

max_element / min_element 정리

범위 안에서 최댓값/최솟값의 위치를 찾는 알고리즘입니다.

사용처 (Where)

- 최댓값 또는 최솟값 찾기
- 값뿐 아니라 위치가 필요할 때
- 점수/길이/크기 비교

방법 (How to Use)

- begin/end 범위를 넘김
- iterator를 반환
- 빈 범위에는 사용하지 않기
- 비교 기준을 lambda로 바꿀 수 있음

기본 정보

- 필요 헤더: `#include <algorithm>`
- 기본 선언: `auto it = max_element(v.begin(), v.end());`

왜 써야 할까? (Why)

- 최댓값 하나 찾으려고 정렬할 필요가 없음
- 위치까지 함께 얻을 수 있음
- 코드가 짧고 의도가 명확함

주요 함수

함수/문법	의미
<code>max_element</code>	최댓값 위치
<code>min_element</code>	최솟값 위치
<code>minmax_element</code>	둘 다 찾기
<code>distance</code>	iterator 위치 계산

기본 코드 예시

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {4, 9, 2, 7};
    auto it = max_element(v.begin(), v.end());

    cout << *it;
}
```

max_element / min_element 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
max/min_element	$O(n)$
값 접근	iterator 역참조
정렬 후 접근	$O(n \log n)$ 라 불필요하게 비쌀 수 있음

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	max_element / min_element 함수 호출만 알면 됨	반복문과 조건을 직접 설계	qsort/직접 반복문 사용
코드 길이	짧고 의도가 분명함	길어지고 실수 지점 증가	자료형 처리와 캐스팅이 번거로움
안정성	표준 라이브러리라 검증됨	구현 실수 가능	포인터/범위 실수 가능
추천 상황	대부분의 일반 상황	원리를 공부하거나 특수 최적화	C 코드와 연동할 때

응용: 가장 긴 문자열 찾기

lambda 비교 함수로 문자열 길이를 기준으로 최댓값을 찾습니다.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<string> words = {"C", "STL", "vector"};
    auto it = max_element(words.begin(), words.end(), [](string a, string b) {
        return a.size() < b.size();
    });
    cout << *it;
}
```

처음 배울 때 자주 하는 실수

- 빈 vector에서 사용 후 *it 하면 위험
- 반환값은 값이 아니라 iterator
- 동점 처리 기준이 필요하면 비교 함수를 명확히 작성

비슷한 항목과 차이

- sort는 전체 순서가 필요할 때
- accumulate는 합계
- minmax_element는 최솟값과 최댓값을 한 번에 찾음

한 줄 정리: `max_element` / `min_element`은(는) '최댓값 또는 최솟값 찾기' 상황에서 먼저 떠올리면 좋은 STL입니다.